

PROGRAM 3:-

AIM

To solve the **Water Jug Problem** using state-space search by transferring water between two jugs to obtain the required amount of water.

ALGORITHM

1. Start the program.
2. Define the capacities of the two jugs.
3. Define the initial state as (0, 0).
4. Generate possible operations:
 - Fill the first jug.
 - Fill the second jug.
 - Empty the first jug.
 - Empty the second jug.
 - Pour water from the first jug into the second jug.
 - Pour water from the second jug into the first jug.
5. Check each new state and avoid already visited states.
6. Continue generating states until the required amount of water is obtained.
7. Display the sequence of states leading to the solution.
8. Stop the program.

PROGRAM

```
from collections import deque

def water_jug(jug1, jug2, target):
    visited = set()
    queue = deque()
    queue.append((0, 0, []))
    while queue:
        x, y, path = queue.popleft()
        if (x, y) in visited:
```

```

        continue
    visited.add((x, y))
    path = path + [(x, y)]
    if x == target or y == target:
        print("\nSolution Steps:")
        for state in path:
            print(state)
        return
    next_states = [
        (jug1, y),
        (x, jug2),
        (0, y),
        (x, 0),
        (max(0, x - (jug2 - y)), min(jug2, x + y)),
        (min(jug1, x + y), max(0, y - (jug1 - x)))
    ]
    for state in next_states:
        if state not in visited:
            queue.append((state[0], state[1], path))
    print("No solution possible.")
jug1 = int(input("Enter capacity of Jug 1: "))
jug2 = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))
water_jug(jug1, jug2, target)

```

OUTPUT:-

```
WaterJugProblem.py
Enter capacity of Jug 1: 5
Enter capacity of Jug 2: 2
Enter target amount: 3
```

Solution Steps:

```
(0, 0)
(5, 0)
(3, 2)
```

RESULT

The Water Jug Problem was successfully solved by applying the possible water-transfer operations and obtaining the required quantity of water.